Lab 1: Introduction to QtSpim

Course: CIS 216 Lab- Computer Organization

MIPS Assembly Language Programming Using QtSpim

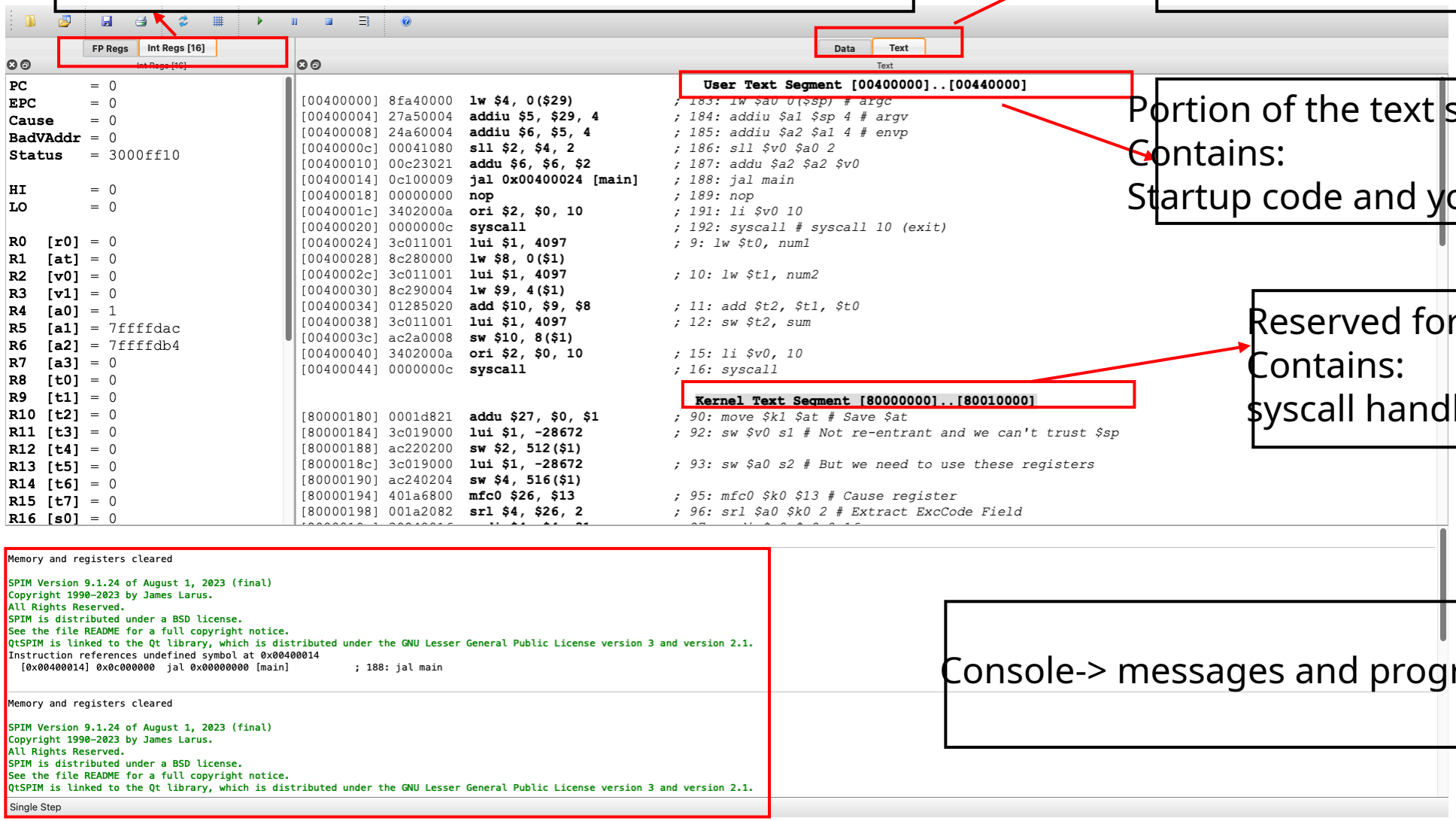
Learning Objectives

- By the end of this lab, students will be able to:
  - Load and run a MIPS program in QtSpim
  - Identify registers, memory, and instructions
  - Explain how data moves: memory -> registers-> ALU->memory

MIPS Assembly Language Programming Using QtSpim

Int Regs holds the integers, addresses (your program)
FP Regs holds the Floating-point values (FP programs)

Data holds variables (your program)
Text holds instructions (your program)



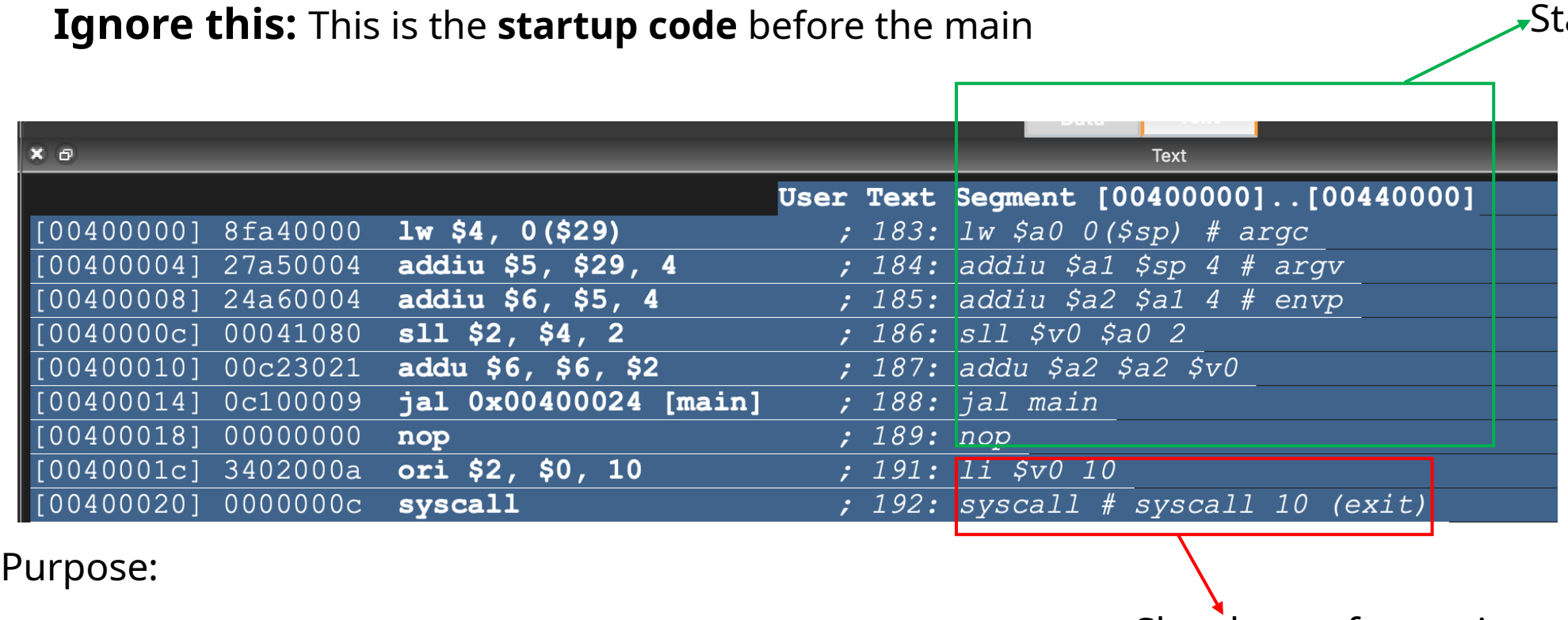
Portion of the text segment for user programs
Contains:
Startup code and your main function

Reserved for OS (kernel code)
Contains:
syscall handlers, exception handling

Console-> messages and program output

MIPS Assembly Language Programming Using QtSpim

Ignore this: This is the **startup code** before the main



Purpose:

- It mimics what a real operating system does:
- Load argc, argv,envp
- Calls main

MIPS Assembly Language Programming Using QtSpim

Ignore this: This is the **kernel code** after the main

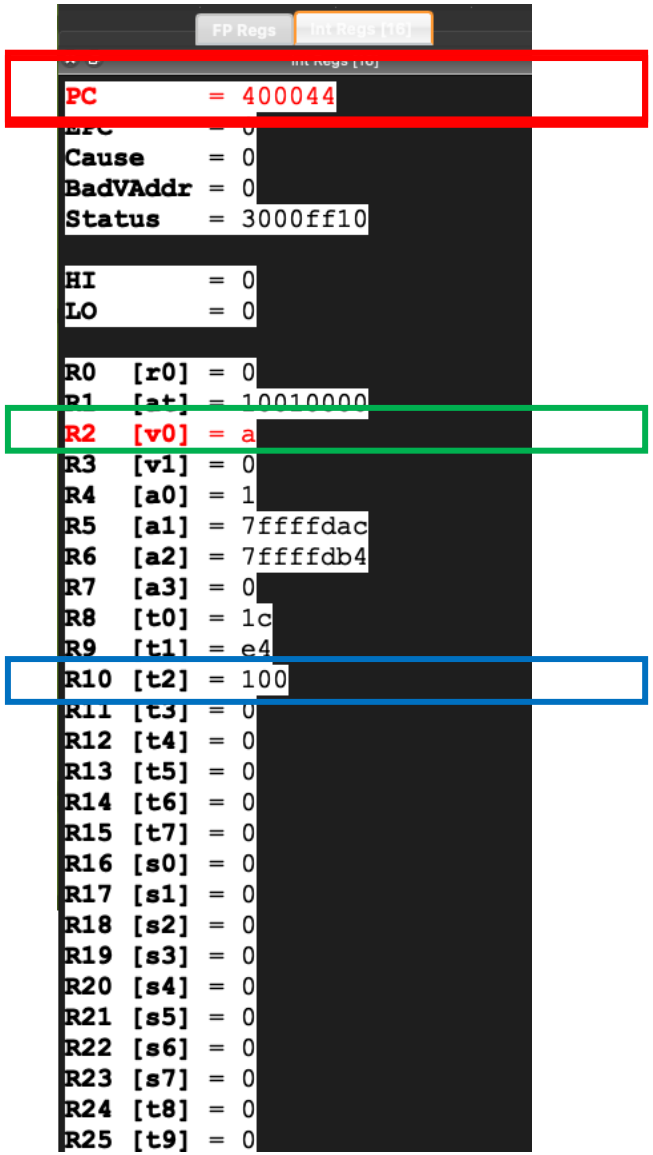
```
                                    Kernel Text Segment [80000000]..[80010000]
[80000180] 0001d821  addu $27, $0, $1          ; 90: move $k1 $at # Save $at
[80000184] 3c019000  lui $1, -28672            ; 92: sw $v0 s1 # Not re-entrant and we can't trust $sp
[80000188] ac220200  sw $2, 512($1)
[8000018c] 3c019000  lui $1, -28672            ; 93: sw $a0 s2 # But we need to use these registers
[80000190] ac240204  sw $4, 516($1)
[80000194] 401a6800  mfc0 $26, $13             ; 95: mfc0 $k0 $13 # Cause register
[80000198] 001a2082  srl $4, $26, 2            ; 96: srl $a0 $k0 2 # Extract ExcCode Field
[8000019c] 3084001f  andi $4, $4, 31           ; 97: andi $a0 $a0 0x1f
[800001a0] 34020004  ori $2, $0, 4             ; 101: li $v0 4 # syscall 4 (print_str)
[800001a4] 3c049000  lui $4, -28672 [__m1_]    ; 102: la $a0 __m1__
[800001a8] 0000000c  syscall                   ; 103: syscall
[800001ac] 34020001  ori $2, $0, 1             ; 105: li $v0 1 # syscall 1 (print_int)
[800001b0] 001a2082  srl $4, $26, 2            ; 106: srl $a0 $k0 2 # Extract ExcCode Field
[800001b4] 3084001f  andi $4, $4, 31           ; 107: andi $a0 $a0 0x1f
[800001b8] 0000000c  syscall                   ; 108: syscall
[800001bc] 34020004  ori $2, $0, 4             ; 110: li $v0 4 # syscall 4 (print_str)
[800001c0] 3344003c  andi $4, $26, 60          ; 111: andi $a0 $k0 0x3c
[800001c4] 3c019000  lui $1, -28672            ; 112: lw $a0 __excp($a0)
[800001c8] 00240821  addu $1, $1, $4
[800001cc] 8c240180  lw $4, 384($1)
[800001d0] 00000000  nop                       ; 113: nop
[800001d4] 0000000c  syscall                   ; 114: syscall
[800001d8] 34010018  ori $1, $0, 24            ; 116: bne $k0 0x18 ok_pc # Bad PC exception requires special checks
[800001dc] 143a0008  bne $1, $26, 32 [ok_pc-0x800001dc]
[800001e0] 00000000  nop                       ; 117: nop
[800001e4] 40047000  mfc0 $4, $14              ; 119: mfc0 $a0 $14 # EPC
[800001e8] 30840003  andi $4, $4, 3            ; 120: andi $a0 $a0 0x3 # Is EPC word-aligned?
[800001ec] 10040004  beq $0, $4, 16 [ok_pc-0x800001ec]
[800001f0] 00000000  nop                       ; 122: nop
[800001f4] 3402000a  ori $2, $0, 10            ; 124: li $v0 10 # Exit on really bad PC
[800001f8] 0000000c  syscall                   ; 125: syscall
[800001fc] 34020004  ori $2, $0, 4             ; 128: li $v0 4 # syscall 4 (print_str)
[80000200] 3c019000  lui $1, -28672 [__m2_]    ; 129: la $a0 __m2__
[80000204] 3424000d  ori $4, $1, 13 [__m2_]
[80000208] 0000000c  syscall                   ; 130: syscall
[8000020c] 001a2082  srl $4, $26, 2            ; 132: srl $a0 $k0 2 # Extract ExcCode Field
[80000210] 3084001f  andi $4, $4, 31           ; 133: andi $a0 $a0 0x1f
[80000214] 14040002  bne $0, $4, 8 [ret-0x80000214]; 134: bne $a0 0 ret # 0 means exception was an interrupt
[80000218] 00000000  nop                       ; 135: nop
```

Purpose:

- Handles syscalls, exceptions, and interrupts
- Inserted by QtSpim, not written by you
- Runs in kernel mode

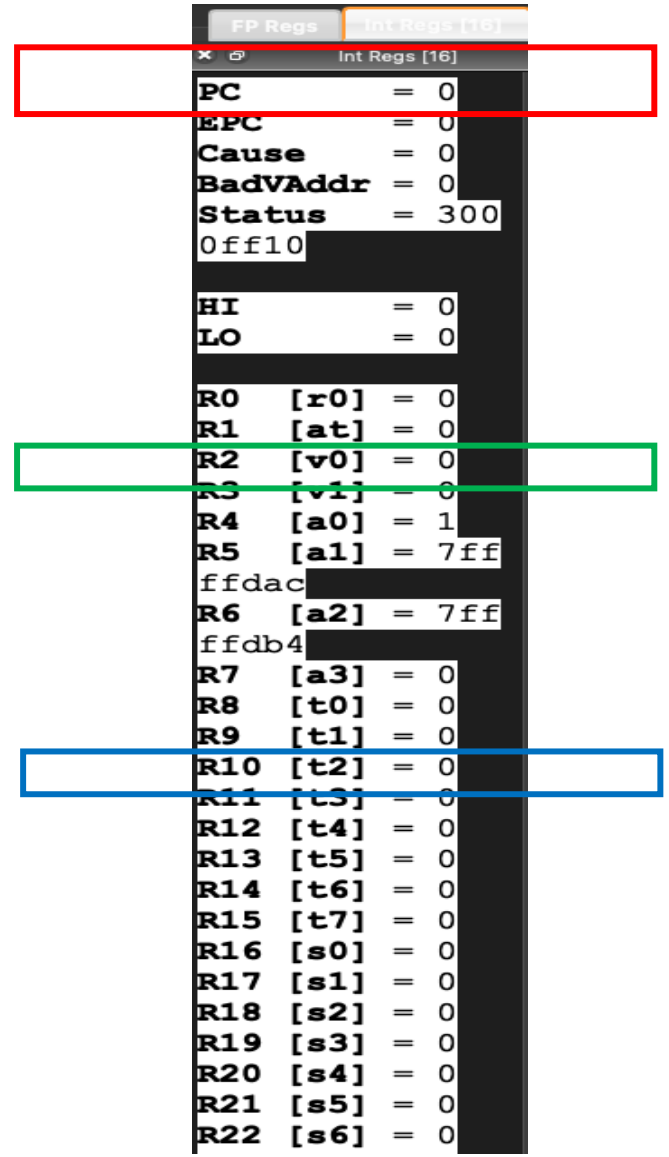
- Saves registers $k0,$k1,$at,$v0,$a0
- Check for syscall, interrupt or bad pc
- Perform requested service
- Print string, print integer, exit
- Restores registers
- Returns user code or terminate

MIPS Assembly Language Programming Using QtSpim

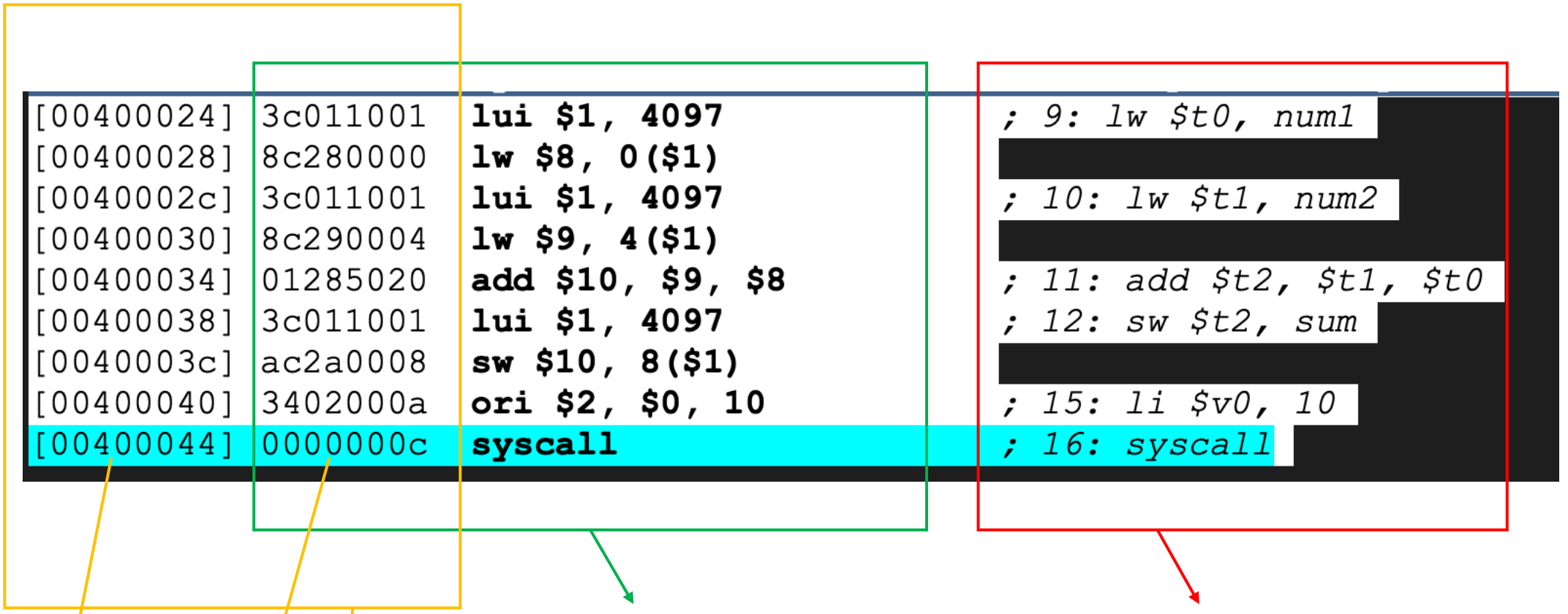


Syscall; exit

0xa (hex) = 10 ; exit syscall



MIPS Assembly Language Programming Using QtSpim



QtSpim executes machine ready code

My Source code

Disassembly view: shows the instruction in memory

The 32 bit machine code (the bits the CPU executes)

[00400044] => the address(PC) of that instruction in memory

Reading QtSpim's Text Segment (Address vs Instruction)

- **Address (PC)**: 0x00400024 → where the instruction lives in memory
- **Machine code**: 0x3c011001 → the 32-bit bits stored at that address
- **Decoded assembly**: lui $1, 4097 → human-readable form of the same bits

Key idea

- The CPU **fetches and executes machine code**.
- QtSpim **decodes** machine code into assembly so humans can read it.

```
[00400024] 3c011001  lui $1, 4097
[00400028] 8c280000  lw $8, 0($1)
[0040002c] 3c011001  lui $1, 4097
[00400030] 8c290004  lw $9, 4($1)
[00400034] 01285020  add $10, $9, $8
[00400038] 3c011001  lui $1, 4097
[0040003c] ac2a0008  sw $10, 8($1)
[00400040] 3402000a  ori $2, $0, 10
[00400044] 0000000c  syscall
```

- **Why 0x3c011001 is 32 bits**
  8 hex digits × 4 bits per hex digit = **32 bits**
  Binary (grouped):
  0011 1100 0000 0001 0001 0000 0000 0001

- **Memory is byte-addressed**
  1 address = 1 byte
  1 MIPS instruction = 4 bytes ⇒ instruction addresses usually increase by **4**:
  …20, …24, …28, …2C

MIPS Assembly Language Programming Using QtSpim

- **Write a MIPS assembly program that loads two integers from memory, adds them, and stores the result back into memory**

```
.data
num1: .word 28
num2: .word 228
sum:  .word 0

.text
.globl main
main:
    lw  $t0, num1      # load num1 into $t0
    lw  $t1, num2      # load num2 into $t1
    add $t2, $t1, $t0  # $t2 = num2 + num1
    sw  $t2, sum       # store result into sum

    li  $v0, 10        # exit
    syscall
```

Tells the assembler that what follows **are variables/constants** stored in memory (RAM). Example: var_a: .word 40 reserves 4 bytes and initializes it to 40.

Tells the assembler that what follows are instructions (machine code) stored in the program's code area.

Makes the label **main visible outside this file** (exported symbol). QtSpim uses it to find where your program starts.

- Initial data (in memory)
  - Num1 = 28
  - Num2 = 228
  - Sum = 0
- Execution (main)
  - Loads num1 into $t0
  - Loads num2 into $t1
  - Adds them: $t2 = $t1 + $t0
- Stores the result into sum
- Exits the program
- Final result
  - Sum = 256

This program loads two values from memory, adds them using the ALU, stores the result back in memory, and exits.

QtSpim Syscall Codes

Syscall code ($v0$)	Service
1	print_int
4	print_string
5	read_int
8	read_string
10	exit
11	print_char
12	read_char
17	exit2

Printing on console:

- move $a0, $t2 #value to print
- li $v0, 1 #print_int
- syscall

```
li $a0, 10 #newline character '\n'
li $v0, 11 #print_char
syscall
```

```
#print_string (syscall 4): print msg
la $a0, msg
li $v0, 4
syscall
```

Student Task

- Task1: Replace num1: .word 5

  num2: .word 7

- Load and run the program in QtSpim
- Step through instructions until sw executes
- Verify result

- Task 2: Add 0xFFFFFFFF and 0x80000000

Hexadecimal Representation

- **num1: .word 5**
- **num2: .word 7**

Verify the result by observing the hexadecimal values in the register

- **Sum = 12**

Register	Hex	Decimal	Mean
$t0	0x1c	28	num1
$t1	0xe4	228	num2
$t2	0x100	256	sum
$t0	0x5	5	num1
$t1	0x7	7	num2
$t2	0xc	12	sum

Hexa	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

How to Identify Registers, Memory & Instructions

- Define in the .data section
  - .data
  - num1: .word 28
  - num2: .word 228
  - sum: .word 0

Memory -> these values live in the memory not in the CPU

- Shown in int Regs ($t0, $t1, $t2, etc)
  - lw $t0, num1   # load memory -> register
  - lw $t1, num2

Registers -> temporarily holds values for computation

- Shown in the Text/User Text Segment
  - add $t2, $t1, $t0

Instructions -> tells the CPU what operation to perform

How Data Moves (Step-by-Step)

Data is loaded from memory into registers, processed by the ALU, and stored back into memory

Memory -> register

- lw $t0, num1 # $t0 =28
- lw $t1, num2. #$t1 = 228

Data is loaded from memory into registers
CPU can’t compute directly on memory

Registers -> ALU

- add $t2, $t1, $t0

ALU adds register values and stores results in another register ($t2 = 256)

Registers -> Memory

Sw $t2, sum

Result is written back to memory

PC Path (Fetch → Next Instruction)

- **PC (Program Counter):** Holds the **address of the next instruction to fetch**.
- **Normal flow (no branch/jump)**
  - **Fetch** instruction at address in PC (e.g., PC = 0x00400020)
  - **Execute** that instruction
  - **Update PC**: PC = PC + 4
    - Next instruction address: 0x00400024



- **If there is a branch/jump**
  - Instead of PC + 4, PC becomes the **target address**:
    - beq/bne (taken) → PC ← branch_target
    - j/jal → PC ← jump_target

- PC is like your “current page pointer”: usually goes to the next page (+4), unless you “jump” to a different page (branch/jump).

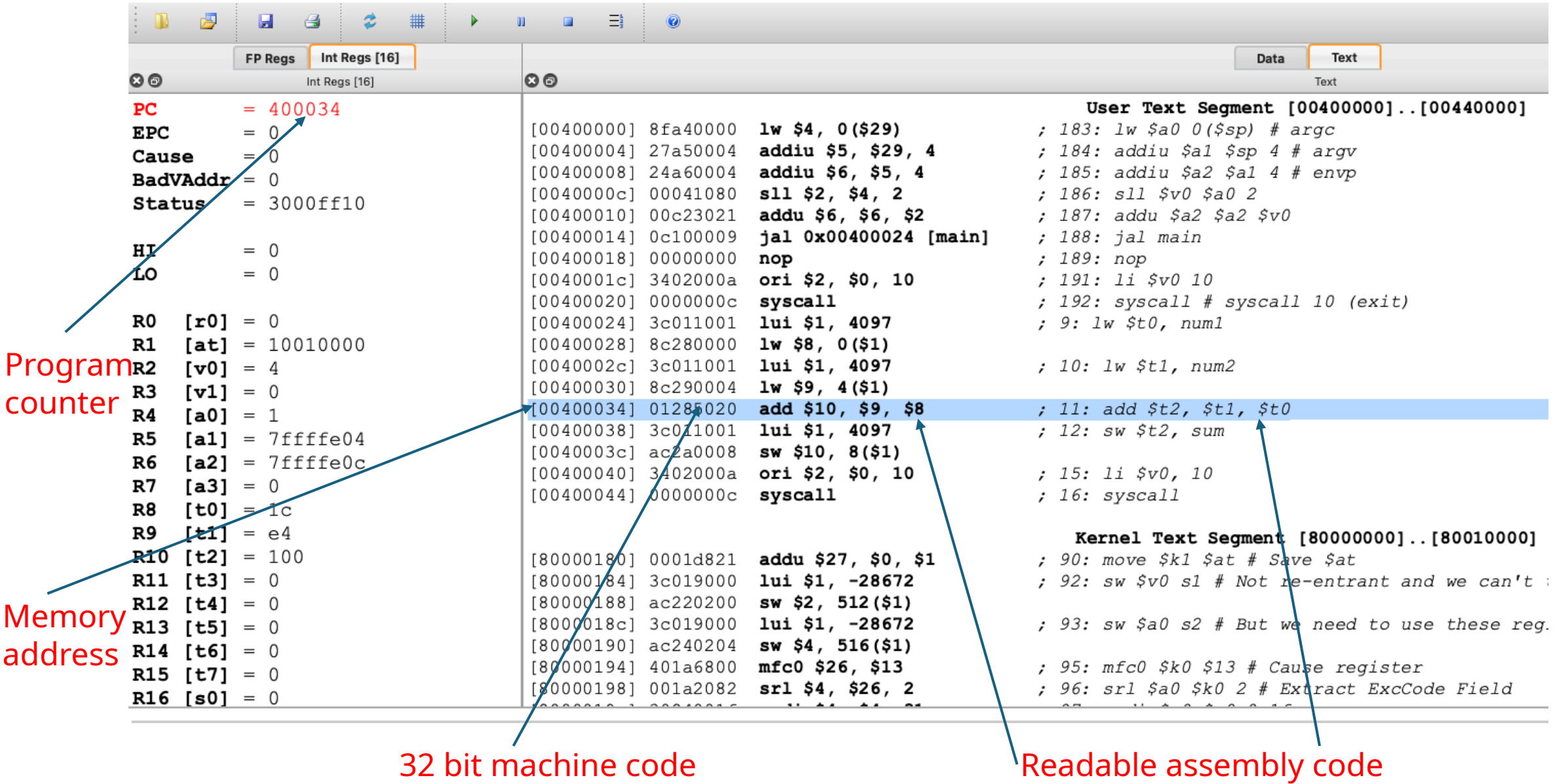
Int Regs: CPU state (special registers + the 32 general-purpose registers)

- **Special registers (top block)**
  - **PC**: Program Counter: address of the next instruction to fetch/execute (0x00400044 here).
  - **EPC**: Exception Program Counter where the CPU was when an exception occurred (0 if none yet).
  - **Cause / BadVAddr / Status**: exception/interrupt bookkeeping registers (treat as OS/exception machinery for now).
  - **HI / LO**: used by multiply/divide instructions (mult/div) to hold results (0 here because you haven’t used them).

The 32 general-purpose registers (R0–R31)

- **R0 [$zero] = 0**: always 0 (hardware-enforced).
- **R1 [$at]**: “assembler temporary” used internally by the assembler ($at = 0x10010000 often holds a base address for data).
- **R2–R3 [$v0,$v1]**: return values / syscall code.
- **$v0 = a** means 0xA (decimal 10): syscall ->exit.
- **R4–R7 [$a0–$a3]**: function arguments (QtSpim also uses these for program startup args).
- **R8–R15 [$t0–$t7]**: temporaries (caller-saved).
- **R16–R23 [$s0–$s7]**: saved registers (callee-saved).
- **R28 [$gp]**: global pointer (used for some global data conventions).
- **R29 [$sp]**: stack pointer (0x7ffffe00 here).
- **R31 [$ra]**: return address (set by jal; here 0x00400018).

Task: Identify the following



32-bit machine code

The CPU executes

The Human reads

- This is the bits the CPU executes
  - Example: 01285020 =>0x01285020
  - QtSpim decodes it to => add $t2, $t1, $t0
- How does this=> 0x01285020 mean this=> add $t2, $t1, $t0

Split R-function layout

OP	rs	rt	rd	shamt	funct
6	5	5	5	5	6

- **op** = 000000 = 0 → R-format
- **rs** = 01001 = 9 → **$t1**
- **rt** = 01000 = 8 → **$t0**
- **rd** = 01010 = 10 → **$t2**
- **shamt** = 00000 = 0 (not used for add)
- **funct** = 100000 = 32 = 0x20 → **add**

0x01285020 =
000000 01001 01000 01010 00000 100000
Is decoded to add $t2, $t1, $t0

Common funct codes(R-format: op=0)

Instruction	funct (bin)	funct (hex)	funct (dec)
add	100000	0x20	32
addu	100001	0x21	33
sub	100010	0x22	34
and	100100	0x24	36
or	100101	0x25	37
slt	101010	0x2A	42
jr	001000	0x08	8
syscall	001100	0x0C	12

- Register (R) format is used for register – register operations (rs, rt, rd)
- Example: add $t2, $t1, $t0

Common opcodes (I/J-format)

Instruction	opcode (bin)	opcode (hex)	opcode (dec)
lw	100011	0x23	35
sw	101011	0x2B	43
beq	000100	0x04	4
bne	000101	0x05	5
addi	001000	0x08	8
andi	001100	0x0C	12
ori	001101	0x0D	13
lui	001111	0x0F	15
j	000010	0x02	2
jal	000011	0x03	3

- Immediate (I) format is used when you need a constant, branch, etc Example addi $t0, $t0, 5
- Jump (J) format is used for far jumps to a target label. Example: j label, jal func

QtSpim

- **Arithmetic:** add, sub, addi
- **Logical:** and, or, xor, nor, andi, ori
- **Memory:** lw, sw
- **Control flow:** beq, bne, j, jal, jr
- **Shift:** sll, srl